

Aleksandar Abrams
Professor David Chiu
CSCI 475 Operating Systems
May 8, 2026

In 1991, a Finnish college student posted to a Usenet forum that he was working on a free operating system. "Just a hobby," he wrote, "won't be big and professional." Today that hobby runs the internet. Linux powers the majority of the world's web servers, all 500 of the world's fastest supercomputers, the Android platform on billions of phones, and an enormous range of embedded systems most people never think about. It is the most successful open-source project in history, and it got there through a combination of good engineering decisions, smart licensing, and a development model that no company has been able to replicate.

This paper looks at how Linux actually works under the hood; how it manages processes and threads, how it schedules CPU time, how it handles memory and why those design choices made it so dominant. It also covers Linux's Virtual File System and the open-source ecosystem that sustains it. It ends on a new challenge that Linux faces in the coming months: California's Digital Age Assurance Act, a 2025 law that requires every operating system provider to build an age-verification API. That sounds straightforward until you realize it applies to volunteer-run Linux distributions that were never designed with anything like that in mind.

Linus Torvalds didn't set out to change computing. He just wanted a UNIX system he could actually use. He was studying computer science at the University of Helsinki, he'd gotten comfortable with UNIX on the university's machines, and he couldn't afford a commercial license for one on his own PC. MINIX existed, but it was restricted to educational use only. So in 1991 he started writing his own kernel from scratch for the Intel 386, the hardware he was using at the time.

What he ended up building didn't contain any AT&T UNIX code. Other UNIX-like projects at the time were getting tangled in licensing disputes, Linux was a clean reimplementation and legally in the clear. He paired it with the GNU Project's existing compilers and utilities, which had been waiting years for a kernel to complete them, and suddenly there was a complete, free operating system that professional developers could actually use for real work. When Torvalds relicensed it under the GPL, which requires anyone distributing a modified version to release their changes under the same terms, contributions started flowing in from developers worldwide. The pace of improvement quickly outstripped what any single team could have managed.

One early decision that attracted criticism was the monolithic kernel architecture. Rather than putting OS services in separate user-space processes the way microkernel designs do, Linux bundles memory management, scheduling, and I/O handling directly into the kernel. Andrew Tanenbaum famously called this obsolete in a 1992 Usenet debate. But the monolithic approach

turned out to be faster in practice, and Linux's module system meant you could still load and unload kernel components at runtime without rebooting. The "wrong" architectural choice worked out way better than expected.

A subtle but important part of Linux's internals is that there is no meaningful distinction between processes and threads at the kernel level. Most OS textbooks treat these two differently, with processes as heavyweight structures with their own address space and threads being their lightweight counterparts that share an address space. In Linux everything is a task, and every task is represented by the same data structure: the `task_struct`.

The `task_struct` holds everything the kernel needs to track an execution context such as scheduling parameters, a pointer to the virtual memory descriptor, the file descriptor table, signal state, saved registers, and the kernel-mode stack [6]. The kernel keeps all live `task_struct`s in a linked list so the scheduler can reach any of them. Each task is assigned a PID; threads within the same process share a thread-group ID (TGID), which is what shows up as the PID in user space.

Linux's task states go a bit further than what's typically covered in an intro OS course. `TASK_RUNNING` means on the CPU or ready to be. But blocked tasks come in two flavors: `TASK_INTERRUPTIBLE`, which can be woken by a signal, and `TASK_UNINTERRUPTIBLE`, which can't. That second state exists for situations where waking early would leave some resource in an inconsistent state such as mid-way through a disk write. There's also `EXIT_ZOMBIE`, which is what a task becomes after calling `exit()` while it waits for its parent to collect the exit code via `wait()`. The distinction between interruptible and uninterruptible sleep is often arbitrary, but when it isn't, it matters a lot.

Linux's scheduler isn't a single algorithm, it's more like a class hierarchy. Real-time tasks registered under `SCHED_FIFO` or `SCHED_RR` always preempt everything else. Background batch jobs use `SCHED_BATCH`, which trades lower wake-up priority for better throughput. Most tasks you'd actually run like terminal sessions, compilers, and browsers go through the Completely Fair Scheduler, class `SCHED_NORMAL`.

CFS was introduced in kernel 2.6.23 in 2007, written by Ingo Molnar, and it works very differently from a classical priority-queue scheduler [4]. The core idea is virtual runtime, or `vruntime`: rather than assigning each task a fixed time slice, the scheduler tracks how much CPU time each task has accumulated in a normalized, priority-weighted form. At every scheduling decision, it picks the task with the smallest `vruntime`: the one that has received the least CPU time relative to its peers. The goal is to approximate what would happen if you had a CPU fast enough to run all tasks truly in parallel.

To make this efficient, runnable tasks are stored in a red-black tree keyed by `vruntime`. The leftmost node is always the next task to run, and it's cached so the scheduler can access it in $O(1)$. When a task gets preempted, its actual elapsed time is multiplied by a weight derived from its nice value and added to its `vruntime`. Nice values run from -20 to +19, where lower means higher priority. For example, a task at nice -20 accumulates `vruntime` slowly, giving it a larger share of the CPU over time, while a task at nice +19 accumulates it fast. This means no separate

priority queues are needed as priority is baked into how vruntime grows.

Timeslice length in CFS isn't fixed either. There's a target latency at about 20ms that defines the window in which every runnable task should get at least one turn. With N tasks running, each gets $1/N$ of that window, weighted by priority. As tasks block and unblock, N changes and the slices adjust. A minimum granularity of around 1ms keeps context switches from dominating when there are a lot of tasks. Starvation is essentially impossible as every task's vruntime eventually becomes the smallest in the tree, guaranteeing it gets scheduled [5].

Every process on Linux gets its own private virtual address space (128 terabytes on a 64-bit x86 system) completely separate from physical RAM, which achieves process isolation. One buggy process can't corrupt another's memory because they're each looking at completely different virtual address spaces.

Getting from a virtual address to a physical one requires walking a four-level page table: Page Global Directory, Page Upper Directory, Page Middle Directory, and Page Table Entry. Each PTE stores the physical frame number plus access control bits (read, write, execute), a present bit, and a dirty bit [7]. Hardware caches recent translations in a TLB so the kernel doesn't have to walk the full table on every memory access like we learned in class. Linux also uses demand paging, loading pages into RAM only when they are needed, just like in class (I'm starting to see a pattern here).

At the physical layer, page frame allocation goes through the Buddy Allocator. It keeps free lists of contiguous 2^k -page blocks for $k = 0$ through 10. Need a four-page block and none are available? It splits an eight-page block and puts the leftover four-page "buddy" on the appropriate list. When a block is freed, the allocator checks whether its buddy is also free and merges them back together, recursively up through the orders. This keeps fragmentation manageable [8]. For smaller allocations, the Slab Allocator sits on top of the buddy system and maintains per-type caches of pre-initialized kernel objects like `task_structs`, inodes, dentries. Getting a new `task_struct` from the slab cache is an $O(1)$ operation; returning it puts it back ready to use without re-running initialization.

When RAM fills up, the page reclaim subsystem starts evicting cold pages from inactive LRU lists to swap. Linux also allows memory overcommit by default. Processes can allocate more virtual memory than physical RAM exists, which works because most processes never actually touch all of their allocated space. When that assumption breaks down badly enough, the OOM killer picks a process to terminate according to a scoring heuristic and frees the memory.

Linux inherits from UNIX the idea that everything is a file. Disk files, directories, devices, sockets, pipes are all accessed through the same file descriptor interface. The mechanism that makes this work is the Virtual File System (VFS), a kernel abstraction layer that defines a standard set of operations (open, read, write, seek, close) that every concrete filesystem implementation has to provide. Whether you're accessing ext4, Btrfs, NFS, or tmpfs doesn't

matter from user space. If you call `read()` you're able to access your file, and the OS handles the rest.

It's hard to overstate what the GPL accomplished. By requiring anyone who distributes a modified version of Linux to release their changes, it created a feedback loop as everyone benefits from everyone else's inputs. Hardware vendors like Intel, AMD, and ARM now maintain their own kernel engineers who upstream architecture and driver support directly, which means Linux usually runs well on new hardware at launch. The Linux Foundation has coordinated this since 2000. No company can do exactly what Linux does, because even if you can provide every feature provided by a global community of developers, you still have to turn a profit. Linux is and always will be 100% free and open source.

In October 2025, Governor Gavin Newsom signed AB 1043, the Digital Age Assurance Act, into law, effective January 1, 2027 [10]. The law requires OS providers to ask users their age at account setup and expose that information to app developers through a real-time API. Developers who receive the signal, which puts users into one of four brackets: under 13, 13–15, 16–17, or 18-and-over, are legally deemed to have actual knowledge of the user's age and bear responsibility for serving age-appropriate content. Penalties go up to \$7,500 per affected child for intentional violations [11].

The law's definition of "operating system provider" is anyone who "develops, licenses, or controls" OS software on any general-purpose computing device. That pulls in corporate operating systems like Windows, macOS, Android, and iOS, but also open source projects like Linux [12]. The bill passed 76-0 in the Assembly and 38-0 in the Senate, unanimously. Which suggests that nobody in the room was thinking about what it means to regulate Linux.

Microsoft and Apple have centralized account systems, legal teams, and the engineering force to build a compliant age-signaling API quickly. Linux on the other hand is run almost entirely by volunteers. A lot of these distributions specifically don't require user accounts as a feature, not an oversight, specifically to promote privacy. Compliance would mean building and indefinitely maintaining an identity-layer API that contradicts Linux's identity.

Linux got to where it is through a series of decisions that look obvious in hindsight but weren't at the time: the monolithic kernel that turned out to be fast and flexible, the GPL that turned a personal project into a global one, the unified task model that simplified scheduling, the CFS that eliminated starvation without needing heuristics, and the four-level page table and buddy allocator that scaled from embedded devices to supercomputers. None of these are flashy. They're just solid engineering choices that compounded over thirty years and have built the foundation for others to improve on.

California's Digital Age Assurance Act is a genuinely hard problem for Linux, not because the goal is wrong but because the law was written for a different kind of OS. Whether the community finds a workable path to compliance, or successfully advocates for an

open-source exemption, will be one of the more interesting regulatory stories in computing over the next few years.

Bibliography

- [1] Torvalds, L. (1991, August 26). "What would you like to see most in minix?" [USENET post]. comp.os.minix. <https://www.cs.cmu.edu/~awb/linux.history.html>
- [2] The Linux Foundation. (2022). "The Linux Foundation: It's not just the Linux operating system." <https://www.linuxfoundation.org/blog/blog/the-linux-foundation-its-not-just-the-linux-operating-system>
- [3] Love, R. (2010). Linux Kernel Development (3rd ed.). Addison-Wesley Professional.
- [4] Linux Kernel Documentation. (n.d.). "CFS Scheduler." <https://docs.kernel.org/scheduler/sched-design-CFS.html>
- [5] Molnár, I. (2007). "Completely fair process scheduling in Linux." Linux Journal. <https://www.linuxjournal.com/node/10267>
- [6] The Linux Documentation Project. (n.d.). "Chapter 4: Processes." <https://tldp.org/LDP/tlk/kernel/processes.html>
- [7] The Linux Documentation Project. (n.d.). "Chapter 3: Memory Management." <https://tldp.org/LDP/tlk/mm/memory.html>
- [8] Linux Kernel Documentation. (n.d.). "Memory Management." <https://docs.kernel.org/admin-guide/mm/index.html>
- [9] Wikipedia. (2025). "History of Linux." https://en.wikipedia.org/wiki/History_of_Linux
- [10] California Legislature. (2025). AB 1043: Age verification signals: software applications and online services. https://leginfo.legislature.ca.gov/faces/billNavClient.xhtml?bill_id=202520260AB1043
- [11] Alston & Bird LLP. (2025, November 10). "California Enacts Digital Age Verification Law." <https://www.alstonprivacy.com/california-enacts-digital-age-verification-law/>
- [12] Corbet, J. (2026, March 11). "California's Digital Age Assurance Act and Linux distributions." LWN.net. <https://lwn.net/Articles/1062112/>

[13] Hruska, J. (2026, March 1). "California introduces age verification law for all operating systems, including Linux and SteamOS." Tom's Hardware.
<https://www.tomshardware.com/software/operating-systems/california-introduces-age-verification-law>